COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/Data Structure Algorithm	**SECTION:**	CPE21S3
DATE PERFORMED:	July 23, 2026	**DATE SUBMITTED:**	July 27, 2026
NAME:	Gresola, Aryanna Keith G.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 3.1 **Linked List**

1. PROCEDURE OUTPUT

ILO A: Construct C++ code for a singly and doubly linked list in C++

Table 3-1. Output of Initial/Simple Implementation

INPUT

HOA3p1 > main.cpp > main()

```cpp
#include <iostream>
#include "simply_LL.h"

int main(){

    //creating empty list
    SimplyList<char>* head = new SimplyList<char>;
    SimplyList<char>* second =new SimplyList<char>;
    SimplyList<char>* third = new SimplyList<char>;
    SimplyList<char>* fourth = new SimplyList<char>;


    //assign the data
    head->data = 'A';
    head->next = second;

    second->data = 'Y';
    second->next = third;

    third->data = 'A';
    third->next = fourth;

    fourth->data = 'G';
    fourth->next = nullptr;
```

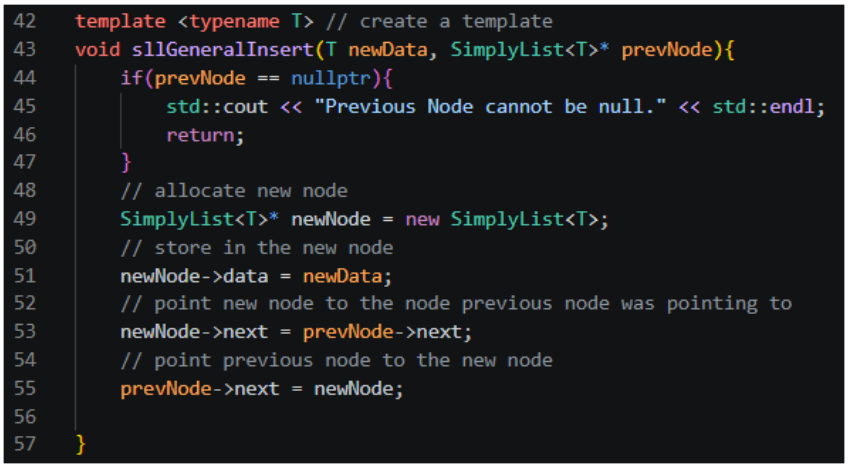
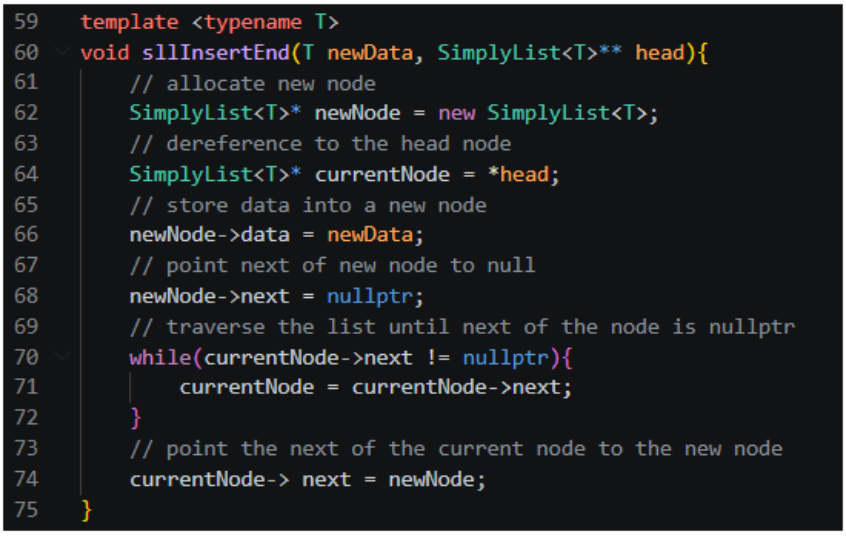
DISCUSSION
The nodes were created and linked together to form a singly linked list with the values A → Y → A → G. Each node contains a data value and a pointer to the next node, while the last node points to nullptr to indicate the end of the list. At this stage, the program only creates the linked list structure in memory and does not perform any operations such as traversal, insertion, or deletion. Therefore, no output is displayed yet.

Table 3-2. Code for the List Operations

OPERATION	SCREENSHOT

Traversal	`template <typename T> // create a template`
	`void listTraversal(SimplyList<T>* head){`
	`  if(head == nullptr){`
	`  std::cout << "List is empty." << std::endl;`
	`  return;`
	`  }`
	
	`  while(head != nullptr){`
	`  std::cout << head->data;`
	`  if(head->next != nullptr){`
	`  std::cout <<" -> ";`
	`  }`
	`  head = head->next;`
	`  }`
	`}`

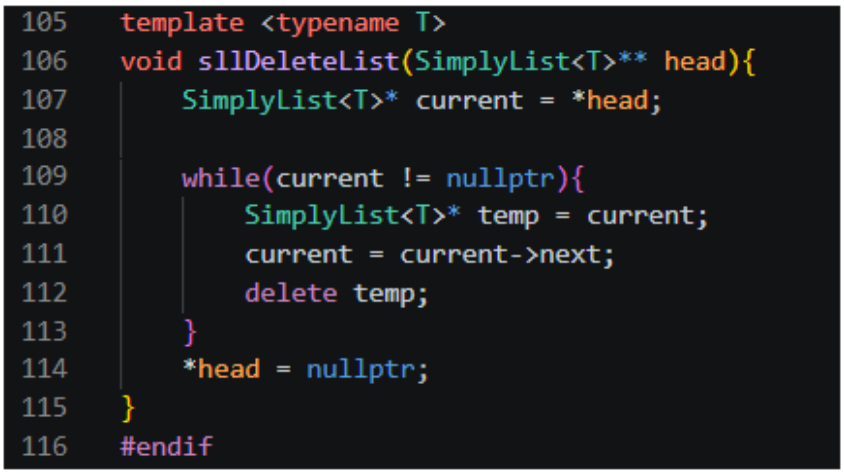
Insertion at head	`// INSERTION`
	`template <typename T> // create a template`
	`void sllinsertHead(T newData, SimplyList<T>** currentHead){`
	`  // allocate memory`
	`  SimplyList<T>* newNode = new SimplyList<T>;`
	`  // put data to new node`
	`  newNode->data = newData;`
	`  // set next of the new node to the previous head`
	`  newNode->next = *currentHead;`
	`  // reset head to point to the new node`
	`  *currentHead= newNode;`
	`}`

Insertion at any part of the list	
Insertion at the end	

```cpp
template <typename T> // create a template
void sllGeneralInsert(T newData, SimplyList<T>* prevNode){
    if(prevNode == nullptr){
        std::cout << "Previous Node cannot be null." << std::endl;
        return;
    }
    // allocate new node
    SimplyList<T>* newNode = new SimplyList<T>;
    // store in the new node
    newNode->data = newData;
    // point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    // point previous node to the new node
    prevNode->next = newNode;

}
```

```cpp
template <typename T>
void sllInsertEnd(T newData, SimplyList<T>** head){
    // allocate new node
    SimplyList<T>* newNode = new SimplyList<T>;
    // dereference to the head node
    SimplyList<T>* currentNode = *head;
    // store data into a new node
    newNode->data = newData;
    // point next of new node to null
    newNode->next = nullptr;
    // traverse the list until next of the node is nullptr
    while(currentNode->next != nullptr){
        currentNode = currentNode->next;
    }
    // point the next of the current node to the new node
    currentNode-> next = newNode;
}
```

Delete of a node	
Delete all list	

```cpp
template <typename T>
void sllDeleteNode(T findData, SimplyList<T>** head){
    // check if there is no list to be deleted
    if(*head == nullptr) return;
    // find previous node of the node to be deleted
    SimplyList<T>* currNode = *head;
    SimplyList<T>* prevNode = nullptr;

    // searching loop
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }

    // data was not found:
    if (currNode == nullptr) return;

    //what if the node to be deleted is the head node?
    if(prevNode == nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }
    // we have to delete the node
    delete currNode;

}
```

```cpp
template <typename T>
void sllDeleteList(SimplyList<T>** head){
    SimplyList<T>* current = *head;

    while(current != nullptr){
        SimplyList<T>* temp = current;
        current = current->next;
        delete temp;
    }
    *head = nullptr;
}
#endif
```

Table 3-3. Code and Analysis for Singly Linked Lists

a. Traverse the list by passing the head of the created list into the function

MAIN

```
// listTraversal(head);
std::cout << "Traversal: " << std::endl;
listTraversal(head);
std::cout << std::endl;
```

SINGLY.H

```
template <typename T> // create a template
void listTraversal(SimplyList<T>* head){
    if(head == nullptr){
        std::cout << "List is empty." << std::endl;
        return;
    }

    while(head != nullptr){
        std::cout << head->data;
        if(head->next != nullptr){
            std::cout <<" -> ";
        }
        head = head->next;
    }
}
```

OUTPUT

```
Traversal:
C -> P -> E -> 0 -> 1 -> 0
```

b. Insert the element 'G' at the start of the list to replace the current node. Output should now show "GCPE101"

MAIN

```
std::cout << "\n Inserting a new node at the head. " << std::endl;
sllinsertHead('G', &head);
listTraversal(head);
std::cout << std::endl;
```

SINGLY.H

```
// INSERTION
template <typename T> // create a template
void sllinsertHead(T newData, SimplyList<T>** currentHead){
    // allocate memory
    SimplyList<T>* newNode = new SimplyList<T>;
    // put data to new node
    newNode->data = newData;
    // set next of the new node to the previous head
    newNode->next = *currentHead;
    // reset head to point to the new node
    *currentHead= newNode;
}
```

OUTPUT

```
 Inserting a new node at the head.
G -> C -> P -> E -> 0 -> 1 -> 0
```

c. Insert an element “E” with the previous node element being “P”. Output should now show “GCPEE101”.

MAIN

```
std::cout << "\n Inserting a new node at the any. " << std::endl;
sllGeneralInsert('E', head->next->next);
listTraversal(head);
std::cout << std::endl;
```

SINGLY.H

```
template <typename T> // create a template
void sllGeneralInsert(T newData, SimplyList<T>* prevNode){
    if(prevNode == nullptr){
        std::cout << "Previous Node cannot be null." << std::endl;
        return;
    }
    // allocate new node
    SimplyList<T>* newNode = new SimplyList<T>;
    // store in the new node
    newNode->data = newData;
    // point new node to the node previous node was pointing to
    newNode->next = prevNode->next;
    // point previous node to the new node
    prevNode->next = newNode;

}
```

OUTPUT

```
 Inserting a new node at the any.
G -> C -> P -> E -> E -> 0 -> 1 -> 0
```

d. Delete the node containing the element C.

MAIN

```
std::cout << "\nDelete C:" << std::endl;
sllDeleteNode('C', &head);
listTraversal(head);
std::cout << std::endl;
```

SINGLY.H

```
template <typename T>
void sllDeleteNode(T findData, SimplyList<T>** head){
    // check if there is no list to be deleted
    if(*head == nullptr) return;
    // find previous node of the node to be deleted
    SimplyList<T>* currNode = *head;
    SimplyList<T>* prevNode = nullptr;

    // searching loop
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }

    // data was not found:
    if (currNode == nullptr) return;

    //what if the node to be deleted is the head node?
    if(prevNode == nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }
    // we have to delete the node
    delete currNode;
```

OUTPUT

```
 Deleting a node:
G -> P -> E -> E -> 0 -> 1 -> 0
```

e. Delete the node containing the element P.

MAIN

```
std::cout << "\nDelete P:" << std::endl;
sllDeleteNode('P', &head);
listTraversal(head);
std::cout << std::endl;
```

SINGLY.H

```
template <typename T>
void sllDeleteNode(T findData, SimplyList<T>** head){
    // check if there is no list to be deleted
    if(*head == nullptr) return;
    // find previous node of the node to be deleted
    SimplyList<T>* currNode = *head;
    SimplyList<T>* prevNode = nullptr;

    // searching loop
    while(currNode != nullptr && currNode->data != findData){
        prevNode = currNode;
        currNode = currNode->next;
    }

    // data was not found:
    if (currNode == nullptr) return;

    //what if the node to be deleted is the head node?
    if(prevNode == nullptr){
        *head = currNode->next;
    }else{
        prevNode->next = currNode->next;
    }
    // we have to delete the node
    delete currNode;

}
```

OUTPUT

```
Delete P:
G -> E -> E -> 0 -> 1 -> 0
```

f. Show the elements in the list. Output should be “GEE101”.

MAIN

```
std::cout << "\nFinal List:" << std::endl;
listTraversal(head);
std::cout << std::endl;
```

OUTPUT

```
Final List:
G -> E -> E -> 0 -> 1 -> 0
```

Table 3-4. Modified Operations for Doubly Linked Lists

1. MODIFICATION

```
//TEMPLATING
template <typename T>
class DoubleList{
    public:
        T data;
        DoubleList<T> *next = nullptr;
        DoubleList<T> *prev = nullptr;

};
```

ANALYSIS

The main change is the addition of the prev pointer. Unlike the singly linked list that only knows the next node, the doubly linked list also keeps track of the previous node. This allows the list to be traversed in both forward and backward directions.

2. MODIFICATION

```
template <typename T>
void dllInsertHead(T newData, DoubleList<T>** currentHead){
    // create a node
    DoubleList<T>* newNode = createNode(newData);
    //new node should point to the current head
    newNode->next = *currentHead;
    // current head should point back to the new
    if(*currentHead != nullptr){
        (*currentHead)->prev = newNode;
    }
    // update the header pointer
    *currentHead = newNode;
}
```

ANALYSIS

In the singly linked list, only the next pointer of the new node needs to be updated. In the doubly linked list, the previous head must also update its prev pointer so that it points back to the new head. This keeps the connections between nodes correct in both directions.

3. MODIFICATION

```
template <typename T>
void dllInsertEnd(T newData, DoubleList<T> *currentHead){

    //create new node
    DoubleList<T> *newNode = createNode(newData);
    //traverse to the last node
    while(currentHead->next != nullptr){
        currentHead = currentHead->next;
    }
    currentHead->next = newNode;
    //connect to the new node
    newNode->prev = currentHead;
}
```

ANALYSIS

Unlike the singly linked list, inserting a node at the end of a doubly linked list requires updating both the next and prev pointers. The current last node's next pointer is set to the new node, while the new node's prev pointer is linked back to the previous last node. This maintains the correct connections between nodes and allows traversal in both forward and backward directions.

4. MODIFICATION

```
template <typename T>
void dllInsert(T newData, DoubleList<T>* currentNode){
    // will check if current node exists
    if(currentNode == nullptr){
        return;
    }
    // create new node
    DoubleList<T>* newNode = createNode(newData);
    // connect new node
    newNode->next = currentNode->next;
    newNode->prev = currentNode;
    // if not inserting in the end
    if(currentNode->next != nullptr){
        currentNode->next->prev = newNode;
    }
    currentNode->next = newNode;
}
```

ANALYSIS

This operation requires more pointer updates compared to the singly linked list. Aside from connecting the new node using the next pointer, the prev pointers of both the inserted node and the succeeding node also need to be updated. This ensures that the links remain correct in both directions.

5. MODIFICATION

```
template <typename T>
void dllDelete(DoubleList<T>** currentHead, DoubleList<T>* deleteNode){
    if(deleteNode == nullptr){
        return;
    }
    // if deleting the head
    if(deleteNode == *currentHead){
        *currentHead = deleteNode->next;
    }
    // Connect previous node
    if(deleteNode->prev != nullptr){
        deleteNode->prev->next = deleteNode->next;
    }
    // Connect next node
    if(deleteNode->next != nullptr){
        deleteNode->next->prev = deleteNode->prev;
    }
    delete deleteNode;

}
```

ANALYSIS

Deleting a node in a doubly linked list is different because both neighboring nodes need to be reconnected. The previous node updates its next pointer, while the next node updates its prev pointer before the node is deleted. This prevents the list from breaking after deletion.

6. MODIFICATION

```
template <typename T>
void dllDeleteAll(DoubleList<T>** currentHead){
    DoubleList<T>* temp;

    while(*currentHead != nullptr){
        temp = *currentHead;
        *currentHead = (*currentHead)->next;
        delete temp;
    }
}
```

ANALYSIS

This operation is almost the same as in the singly linked list since each node is deleted one by one. Even though each node has a prev pointer, no extra updates are needed because the entire list is being removed.

7. MODIFICATION

```
    // Forward Traversal
    std::cout << "Forward Traversal: " << std::endl;
    // moves toward the last node
    while(currentNode != nullptr){
        // print the data
        std::cout << currentNode->data<<" ";
        //saved the current node as the last visited node
        tail = currentNode;
        // move to the next node
        currentNode = currentNode->next;
    }
    std::cout << std::endl;

    //backward traversing:
    std::cout << "Backward: " << std::endl;
    while(tail != nullptr){
        //printed the data
        std::cout << tail->data << " ";
        tail = tail->prev;
    }
    std::cout << std::endl;
}
```

ANALYSIS

One advantage of the doubly linked list is that it supports both forward and backward traversal. By using the prev pointer, the program can easily move back through the list without restarting from the head, making traversal more flexible than in the singly linked list.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++
Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

INPUT (CIRCULAR_LL.H)

CIRCULAR_LL.h > addSongToBeginning<T>(T, T, Playlist<T>**)

```cpp
#ifndef CIRCULAR_LL_H
#define CIRCULAR_LL_H

#include <iostream>
#include <string>

// playlist node

template <typename T>
class Playlist{
public:
    T songTitle;
    T artist;
    Playlist<T>* next = nullptr;

};

// creating a new song
template <typename T>
Playlist<T>* createSong(T songTitle, T artist){
    Playlist<T>* newSong = new Playlist<T>;
    newSong->songTitle = songTitle;
    newSong->artist = artist;
    newSong->next = newSong;

    return newSong;
}

// playing all song (traversal)
template <typename T>
void playAllSongs(Playlist<T>* playlistHead){
    if(playlistHead == nullptr){
        std::cout << "The playlist is empty." << std::endl;
        return;
    }
    Playlist<T>* currentSong = playlistHead;
    int songNumber = 1;
    std::cout << "CURRENT PLAYLIST" << std::endl;
    std::cout << "-----------------------" << std::endl;
```

```
40
41      do{
42          std::cout << songNumber << ". "
43                    << currentSong->songTitle << " - " << currentSong->artist << std::endl;
44
45          currentSong = currentSong->next;
46          songNumber++;
47
48      }while(currentSong != playlistHead);
49      std::cout << "-----------------------" << std::endl;
50  }
51
52  // inserting song (head)
53  template <typename T>
54  void addSongToBeginning(T songTitle, T artist, Playlist<T>** playlistHead){
55      Playlist<T>* newSong = createSong(songTitle, artist);
56
57      if(*playlistHead == nullptr){
58          *playlistHead = newSong;
59          return;
60      }
61
62      Playlist<T>* lastSong = *playlistHead;
63      while(lastSong->next != *playlistHead){
64          lastSong = lastSong->next;
65
66      }
67
68      newSong->next = *playlistHead;
69      lastSong->next = newSong;
70
71      *playlistHead = newSong;
72  }
73
74  // inserting song (end)
75
76  template <typename T>
77  void addSongToEnd(T songTitle, T artist, Playlist<T>* playlistHead){
78      if(playlistHead == nullptr)
79      return;
```

```
80
81          Playlist<T>* newSong = createSong(songTitle, artist);
82          Playlist<T>* lastSong = playlistHead;
83
84          while(lastSong->next != playlistHead){
85              lastSong = lastSong->next;
86          }
87
88          lastSong->next = newSong;
89          newSong->next = playlistHead;
90      }
```

```cpp

// inserting song (anywhere)

template <typename T>
void addSongAny(T songTitle, T artist, Playlist<T>* currentSong){
    if(currentSong == nullptr)
    return;

    Playlist<T>* newSong = createSong(songTitle, artist);

    newSong->next = currentSong->next;
    currentSong->next = newSong;
}

// deleting song

template <typename T>
void deleteSong(Playlist<T>** playlistHead, Playlist<T>* songToDelete){
    if(*playlistHead == nullptr || songToDelete == nullptr)
        return;

    // only one song
    if((*playlistHead)->next == *playlistHead){
        delete *playlistHead;
        *playlistHead = nullptr;
        return;
    }

    Playlist<T>* previousSong = nullptr;
    Playlist<T>* currentSong = *playlistHead;

    do{
        if(currentSong == songToDelete){
            if(previousSong == nullptr){
                Playlist<T>* lastSong = *playlistHead;

                while(lastSong->next != *playlistHead){
                    lastSong = lastSong->next;
                }
                *playlistHead = currentSong->next;
                lastSong->next = *playlistHead;
            }
            else{
                previousSong->next = currentSong->next;
            }
            delete currentSong;
            return;
        }
        previousSong = currentSong;
        currentSong = currentSong->next;
    }while(currentSong != *playlistHead);
```

```
142
143     std::cout << "Song not found." << std::endl;
144 }
145
146 // play next song
147
148 template <typename T>
149 void nextSong(Playlist<T>* playlistHead, T currentTitle){
150     if(playlistHead == nullptr)
151         return;
152     Playlist<T>* currentSong = playlistHead;
153
154     do{
155         if(currentSong->songTitle == currentTitle){
156             std::cout << "Current Song : " << currentSong->songTitle << std::endl;
157             std::cout << "Next Song    : " << currentSong->next->songTitle << std::endl;
158             return;
159         }
160         currentSong = currentSong->next;
161     }while(currentSong != playlistHead);
162     std::cout << "Song not found." << std::endl;
163 }
164
165 // play previous song
166 template <typename T>
167 void previousSong(Playlist<T>* playlistHead, T currentTitle){
168     if(playlistHead == nullptr)
169         return;
170
171     Playlist<T>* previousSongNode = nullptr;
172     Playlist<T>* currentSong = playlistHead;
173
174     do{
175
176         if(currentSong->songTitle == currentTitle){
177             if(previousSongNode == nullptr){
178                 Playlist<T>* lastSong = playlistHead;
179
180                 while(lastSong->next != playlistHead){
181                     lastSong = lastSong->next;
182                 }
183
184                 std::cout << "Current Song  : " << currentSong->songTitle << std::endl;
185
186                 std::cout << "Previous Song : " << lastSong->songTitle << std::endl;
187             }else{
188                 std::cout << "Current Song  : " << currentSong->songTitle << std::endl;
189                 std::cout << "Previous Song : " << previousSongNode->songTitle << std::endl;
190             }
191             return;
192         }
```

```
193
194             previousSongNode = currentSong;
195             currentSong = currentSong->next;
196
197         }while(currentSong != playlistHead);
198         std::cout << "Song not found." << std::endl;
199     }
200
201     // clear playlist
202     template <typename T>
203     void clearPlaylist(Playlist<T>** playlistHead){
204         if(*playlistHead == nullptr)
205             return;
206
207         Playlist<T>* currentSong = (*playlistHead)->next;
208
209         while(currentSong != *playlistHead){
210
211             Playlist<T>* temp = currentSong;
212             currentSong = currentSong->next;
213             delete temp;
214         }
215
216         delete *playlistHead;
217         *playlistHead = nullptr;
218
219         std::cout << "Playlist cleared." << std::endl;
220     }
221
222     #endif
```

INPUT (MAIN.CPP)

main.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "CIRCULAR_LL.h"

int main() {

    // create the initial playlist
    Playlist<std::string>* playlistHead =
        createSong(std::string("It Might Be You"), std::string("Stephen Bishop"));

    Playlist<std::string>* secondSong =
        createSong(std::string("I'll Make Love To You"), std::string("Boyz II Men"));

    Playlist<std::string>* thirdSong =
        createSong(std::string("Bakit Ngayon"), std::string("Julie Anne San Jose"));

    // link the songs into a circular playlist
    playlistHead->next = secondSong;
    secondSong->next = thirdSong;
    thirdSong->next = playlistHead;

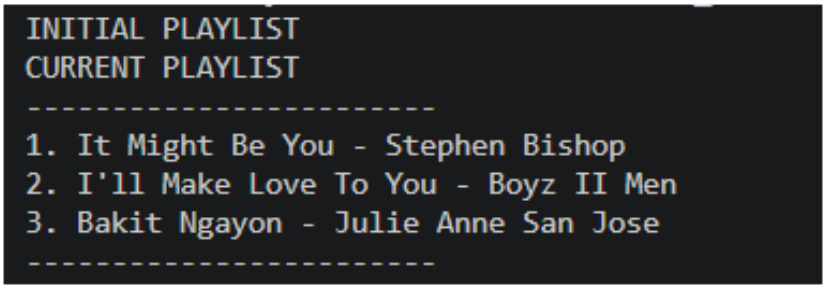
    // display the initial playlist
    std::cout << "INITIAL PLAYLIST" << std::endl;
    playAllSongs(playlistHead);
    std::cout << std::endl;

    // insert song at the beginning
    std::cout << "INSERT SONG AT THE BEGINNING" << std::endl;
    addSongToBeginning(
        std::string("Residuals"),
        std::string("Chris Brown"),
        &playlistHead
    );
    playAllSongs(playlistHead);
    std::cout << std::endl;

    // insert song at the end
    std::cout << "INSERT SONG AT THE END" << std::endl;
    addSongToEnd(
        std::string("The Visitor"),
        std::string("SIENNA SPIRO"),
        playlistHead
    );
    playAllSongs(playlistHead);
    std::cout << std::endl;
```

```
46
47      // insert song after a specific song
48      std::cout << "INSERT SONG AFTER I'LL MAKE LOVE TO YOU" << std::endl;
49      addSongAny(
50          std::string("lowkey"),
51          std::string("NIKI"),
52          secondSong
53      );
54      playAllSongs(playlistHead);
55      std::cout << std::endl;
56
57      // next song testing
58      std::cout << "NEXT SONG TESTING" << std::endl;
59      nextSong(playlistHead, std::string("Bakit Ngayon"));
60      std::cout << std::endl;
61
62      // previous song testing
63      std::cout << "PREVIOUS SONG TESTING" << std::endl;
64      previousSong(playlistHead, std::string("Bakit Ngayon"));
65      std::cout << std::endl;
66
67      // delete song testing
68      std::cout << "DELETE SONG TESTING" << std::endl;
69      deleteSong(&playlistHead, secondSong);
70      playAllSongs(playlistHead);
71      std::cout << std::endl;
72
73      // clear playlist
74      std::cout << "CLEAR PLAYLIST TESTING" << std::endl;
75      clearPlaylist(&playlistHead);
76      playAllSongs(playlistHead);
77
78      return 0;
79  }
```

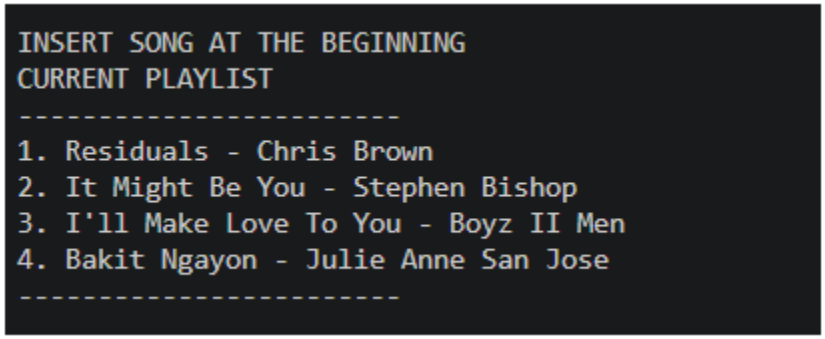
OUTPUT (INITIAL PLAYLIST)



ANALYSIS

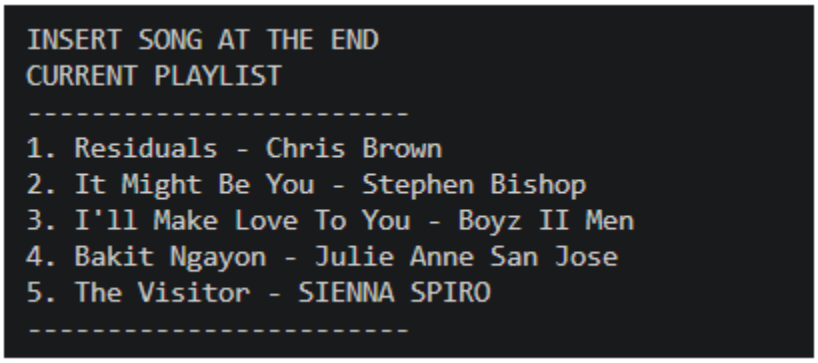
This output shows that the initial playlist was created successfully. The songs were linked correctly, and the program displayed each song once, showing that the circular linked list was set up properly.

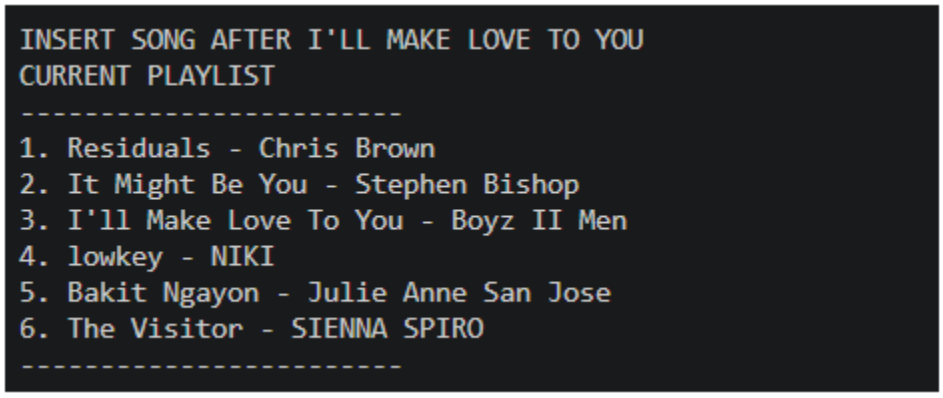
OUTPUT (INSERT SONG AT THE BEGINNING)



ANALYSIS

This output shows that a new song was successfully added at the beginning of the playlist.

OUTPUT (INSERT SONG AT THE END)

ANALYSIS
This output shows that a new song was successfully added at the end of the playlist.

OUTPUT (INSERT SONG AFTER SPECIFIC SONG)

ANALYSIS
This output shows that a song was inserted after the selected song.

OUTPUT (NEXT AND PREVIOUS TESTING)

```
NEXT SONG TESTING
Current Song : Bakit Ngayon
Next Song    : The Visitor

PREVIOUS SONG TESTING
Current Song  : Bakit Ngayon
Previous Song : lowkey
```

ANALYSIS

These outputs show that the playlist can navigate both forward and backward.

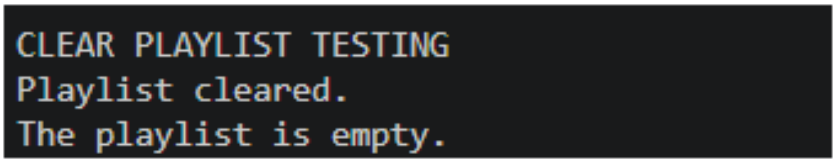
OUTPUT (DELETE A CERTAIN SONG)

```
DELETE SONG TESTING
CURRENT PLAYLIST
------------------------
1. Residuals - Chris Brown
2. It Might Be You - Stephen Bishop
3. lowkey - NIKI
4. Bakit Ngayon - Julie Anne San Jose
5. The Visitor - SIENNA SPIRO
------------------------
```

ANALYSIS

This output shows that the selected song was removed successfully.

OUTPUT (CLEARING THE PLAYLIST)



ANALYSIS

This output shows that all songs were removed from the playlist.

GENERAL ANALYSIS

The program successfully implements a circular linked list to simulate a music player playlist. It first creates a playlist by linking multiple songs together in a circular structure, where the last song points back to the first song instead of nullptr. This circular connection is demonstrated in the playAllSongs() function, which uses a do-while loop to traverse the playlist and stops only when it reaches the head node again. As a result, every song is displayed exactly once while showing the looping behavior of a circular linked list.

The insertion functions correctly add songs at the beginning, end, or after a specific song without breaking the circular connections between the nodes. Likewise, the deletion function removes the selected song while reconnecting the remaining songs, ensuring that the playlist remains complete and circular after the operation. The nextSong() and previousSong() functions further demonstrate playlist navigation by correctly identifying the songs before and after the selected song, showing that traversal works properly in both directions despite using a singly circular linked list. Lastly, the clearPlaylist() function successfully deletes all nodes in the playlist and resets the head pointer to nullptr, which is verified by the output indicating that the playlist is empty.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Code assistance, explanation of concepts, and grammar correction.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: ChatGPT was used for code assistance, implementation suggestions, debugging, concept explanations, and grammar improvements. It did not replace my original thinking, analysis, authorship, or completion of the required work.

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.